React Router – Navigation & Dynamic Routes

(Creating Multi-Page React Apps without Full Page Reload)

📌 1. What is React Router?

React Router is a library used in React to enable **navigation between different components (pages)** without reloading the entire page.

👉 In traditional websites:

- Clicking a link → full page reload

👉 In React (SPA – Single Page Application):

- Clicking a link → only specific component updates (NO reload)

📌 2. Why Use React Router?

Without React Router:

- You only have **one page**
- You manually show/hide components

With React Router:

- You can create **multiple pages**
- URL changes without refreshing
- Better user experience 🚀

📌 3. Install React Router

```
npm install react-router-dom
```

📌 4. Basic Concepts

◇ 1. BrowserRouter

Wraps your entire app and enables routing.

```
import { BrowserRouter } from "react-router-dom";

<BrowserRouter>
  <App />
</BrowserRouter>
```

◇ 2. Routes

Container that holds all route definitions.

```
import { Routes, Route } from "react-router-dom";

<Routes>
  <Route path="/" element={<Home />} />
</Routes>
```

◇ 3. Route

Defines a path and which component to render.

```
<Route path="/about" element={<About />} />
```

👉 Means:

- If URL = `/about` → show `<About />`

📌 5. Creating a Multi-Page App

Example Structure:

```
src/
 ├── pages/
 │     ├── Home.jsx
 │     ├── About.jsx
 │     └── Contact.jsx
 ├── App.jsx
 ├── main.jsx
```

◇ main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
```

```
import App from "./App";
import { BrowserRouter } from "react-router-dom";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

◇ App.jsx

```
import { Routes, Route } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";

function App() {
  return (
    <>
      <h1>My Website</h1>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </>
  );
}

export default App;
```

📌 6. Navigation using Link

Instead of `<a>` tag ❌ (causes reload)

Use **Link** ☑

```
import { Link } from "react-router-dom";

<Link to="/">Home</Link>
<Link to="/about">About</Link>
<Link to="/contact">Contact</Link>
```

👉 Why NOT `<a>`?

- `<a>` → reloads page
- `<Link>` → updates UI without reload

📌 7. Navigation using useNavigate (Programmatic Navigation)

Used when navigation happens after some action (like form submit).

```
import { useNavigate } from "react-router-dom";

function Login() {
  const navigate = useNavigate();

  const handleLogin = () => {
    // after login
    navigate("/dashboard");
  };

  return <button onClick={handleLogin}>Login</button>;
}
```

📌 8. Dynamic Routes (Very Important ⭐)

Dynamic routes allow you to pass **parameters in URL**.

◇ Example Scenario:

You have a **User List**, and when clicking a user → open their profile.

Step 1: Define Dynamic Route

```
<Route path="/user/:id" element={<UserProfile />} />
```

👉 `:id` is a **dynamic parameter**

Step 2: Navigate with Dynamic Value

```
<Link to="/user/1">User 1</Link>
<Link to="/user/2">User 2</Link>
```

Step 3: Access Parameter using useParams

```
import { useParams } from "react-router-dom";

function UserProfile() {
```

```
  const { id } = useParams();

  return <h2>User ID: {id}</h2>;
}
```

👉 If URL = `/user/5`
→ Output: `User ID: 5`

📌 9. Nested Routes (Optional but Important)

Used when pages have sub-pages.

Example:

```
<Route path="/dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>
```

In Dashboard Component:

```
import { Outlet } from "react-router-dom";

function Dashboard() {
  return (
    <>
      <h1>Dashboard</h1>
      <Outlet />
    </>
  );
}
```

👉 `<Outlet />` renders child routes

📌 10. 404 Page (No Match Route)

```
<Route path="*" element={<h1>Page Not Found</h1>} />
```

👉 If route not found → show error page

📌 11. Key Differences (Important for Exams)

Feature	Traditional Website	React Router
Page Reload	Yes	No
Speed	Slower	Faster
UX	Less smooth	Smooth
Navigation	`<a>` tag	`<Link>`

📌 12. Real-World Scenario

🛒 E-commerce Website

Routes:

- `/` → Home
- `/products` → Product List
- `/product/:id` → Product Detail
- `/cart` → Cart Page

👉 Dynamic route:

```
/product/101
/product/202
```

📌 13. Common Mistakes ⚠️

❌ Using `<a>` instead of `<Link>`
❌ Forgetting `BrowserRouter`
❌ Wrong path syntax (`/user/id` instead of `/user/:id`)
❌ Not using `useParams()` correctly

📌 14. Summary

- React Router enables **SPA navigation**
- No full page reload
- Main components:
  - `BrowserRouter`
  - `Routes`
  - `Route`
  - `Link`
- Dynamic routes use `:param`
- Access params using `useParams()`

☑ Practice Task

Create a mini app:

- Home Page
- Products Page
- Product Detail Page (Dynamic Route)

👉 URL Example:

```
/product/1
/product/2
```